

Экзамен

1. Архитектура компьютерных систем. Архитектура Фон-Неймана и Гарвардская архитектура. Принципы архитектуры Фон-Неймана. Архитектуры NUMA и UMA.

Архитектура компьютерных систем в основном делится на два типа:

Гарвардскую и архитектуру Фон-Неймана.

В Гарвардской архитектуре существует **раздельная память для команд и для данных**, а также **раздельные шины** для их передачи.

В архитектуре Фон-Неймана **команды и данные хранятся в одной общей памяти** и передаются по **одной общей шине**, из-за чего одновременно читать команду и данные невозможно — возникает так называемое **узкое место Фон-Неймана**.

Принципы архитектуры Фон-Неймана:

- **Однородность памяти** — команды и данные хранятся вместе и не различаются по типу.
- **Адресность** — каждая ячейка памяти имеет адрес.
- **Программное управление** — вычисления выполняются как последовательность инструкций программы.
- **Двоичное кодирование** — команды и данные представлены в двоичном виде.

Также различают архитектуры организации доступа процессоров к памяти: **UMA** и **NUMA**.

В архитектуре **UMA (Uniform Memory Access)** все процессоры имеют доступ к **общей оперативной памяти с одинаковым временем задержки** через общую шину. Это просто, но при увеличении числа процессоров возникает конкуренция за шину и система плохо масштабируется.

В архитектуре **NUMA (Non-Uniform Memory Access)** память распределена по узлам:

каждый процессор имеет **свою локальную память**, доступ к которой быстрее, чем к памяти других узлов. Доступ к удалённой памяти возможен, но медленнее. Такая архитектура хорошо масштабируется, но требует поддержки со стороны операционной системы и учёта привязки процессов к памяти (memory affinity).

2. Общая организация процессора, памяти и вычислений

1) Организация процессора

Процессор состоит из **ядер**, которые выполняют инструкции.

Внутри ядра есть:

- **Регистры** — самая быстрая память для временных данных.
- **Кэши L1/L2/L3**
 - **L1** делится на **I\$ (инструкции)** и **D\$ (данные)** → можно читать параллельно.
 - **L2** больше и медленнее L1.
 - **L3** общий для всех ядер и самый большой из кэшей.
- **TLB** — хранит быстрые соответствия “виртуальный адрес → физический”, чтобы не лазить постоянно в таблицы страниц.
- **АЛУ / FPU / SIMD** — блоки вычислений (обычные, float, векторные).
- **Конвейер** — выполнение команды разбито на этапы, чтобы разные инструкции шли параллельно по стадиям.
- **Branch predictor** — предсказывает переходы, чтобы не сбрасывать конвейер.
- **Write buffer** — буфер, который ускоряет запись в память (процессор не ждёт RAM).

2) Организация памяти

Память делается **иерархией**: чем выше — тем быстрее и меньше.

Регистры → L1 → L2 → L3 → RAM → диск

Виртуальная память: способ организации памяти, где каждый процесс имеет свое адресное пространство, думая, что у него есть своя независимая память.

Виртуальные адреса преобразуются в физические при помощи таблиц трансляций

3) Организация вычислений

Ядро выполняет каждую команду последовательно

- Выборка команды (Instruction Fetch)
- Декодирование инструкции (Instruction Decode)
- Исполнение (Execution)
- Чтение памяти (MEM)
- Запись (Write Back)

3. Прерывания: организация, типы, контроллер прерываний

Прерывания — сигнал процессору о событии, требующем внимания. Процессор сохраняет контекст и переходит в обработчик прерывания.

Типы прерываний:

- **Асинхронные (аппаратные)** — от внешних устройств: таймер, мышь, клавиатура, диск, сеть (IRQ).
- **Синхронные (исключения)** — возникают при выполнении команды: деление на 0, неправильная инструкция, page fault, обращение к запрещённой памяти.

Контроллер прерываний — принимает IRQ (аппаратный запрос на прерывание), расставляет приоритеты и маршрутизирует прерывания на CPU/ядро.

- **PIC** — старый контроллер, ограниченное число линий (≈ 8), расширяли каскадом master/slave.
- **APIC** — современный вариант для многоядерных систем:
 - устройства → **IO APIC** → дальше прерывание отправляется на нужное ядро через **LAPIC**
- **PCIe + MSI** — прерывание идёт не по отдельному проводу, а через **запись в память (message)**.

4. Типичные функции ОС. Интерфейсы ОС. ОС как замена оператора ЭВМ.

ОС — определение

Операционная система — системное ПО, которое **управляет ресурсами** (CPU, память, устройства, файлы) и даёт **интерфейс** пользователю и программам, скрывая детали железа.

Типичные функции ОС

- **Управление процессами**: создание/завершение, планирование, переключение.
- **Управление памятью**: выделение/освобождение, виртуальная память, защита.
- **Файловая система**: файлы/каталоги, доступ, права.
- **Ввод-вывод**: драйверы, буферизация, унификация доступа к устройствам.
- **Защита и изоляция**: права, разграничение доступа, защита памяти.
- **Обработка ошибок и учет**: логирование, диагностика, статистика ресурсов.

Интерфейсы ОС

- **ISA** — интерфейс между аппаратным обеспечением и машинным кодом, определяет

набор команд процессора.

- **ABI** — интерфейс между скомпилированной программой и операционной системой, определяет формат исполняемых файлов, системные вызовы, соглашения о вызовах функций.
- **API** — интерфейс для программиста, набор функций и библиотек, через которые прикладные программы обращаются к службам ОС.

ОС как замена оператора ЭВМ

ОС автоматизирует то, что делал оператор:

- принимает задания (программы),
- загружает/запускает,
- распределяет ресурсы,
- контролирует выполнение и ошибки,
- выдаёт результат.

5) Пакетная обработка. Системный монитор

Цель

Минимизация машинного времени, так как вычислительное время ЭВМ дорогое, простой недопустим.

Системный монитор

Системный монитор — это дополнительное ПО, которое заранее загружалось в ЭВМ и обеспечивало **автоматический последовательный запуск программ** без постоянного участия оператора.

Состав системного монитора

- **Обработчики прерываний** — сигнализируют о завершении или сбое выполнения программы.
- **Драйверы устройств** — обеспечивают работу с устройствами ввода-вывода.
- **Планировщик заданий** — по сигналу прерывания передаёт управление следующему заданию.
- **Интерпретатор командного языка** — управляет выполнением заданий.

Пакетная обработка

Пакетная обработка — способ работы ЭВМ, при котором несколько программ объединяются в **пакет** и выполняются **последовательно под управлением системного**

монитора.

Принцип работы пакетной обработки

- Оператор формирует пакет заданий и загружает его в устройство ввода.
- Монитор считывает задание из пакета и передаёт ему управление.
- После завершения или ошибки управление возвращается монитору.
- Монитор запускает следующее задание.
- После выполнения всего пакета результаты выводятся на устройство вывода.

6) Анализ общесистемной эффективности, как предусловие многозадачности. Многозадачность, как способ повышения системной эффективности. Системы разделения времени

Анализ общесистемной эффективности как предусловие многозадачности:

Одно задание часто даёт низкую загрузку CPU, потому что вычисляет быстро, но много времени проводит в **ожидании ввода-вывода (I/O)**. Анализ показывает, что теряется машинное время из-за простоев. Решение — держать несколько задач и переключаться между ними: пока одни ждут I/O, другие вычисляют.

Многозадачность как способ повышения системной эффективности:

Это **мультипрограммирование**: когда одна задача ждёт I/O, другая использует CPU → простой уменьшается, загрузка CPU ближе к постоянной → общая эффективность/пропускная способность системы растёт.

Системы разделения времени:

Это вид многозадачности для интерактивной работы: каждому процессу/пользователю выдаётся **квант времени**, а переключение происходит по **таймерному прерыванию** (планировщик). Один пользователь использует систему неэффективно, но группа пользователей оптимального размера повышает общую загрузку и создаёт ощущение “одновременной” работы.

7) Процессы, проблемы современных процессов. Планирование выполнения процессов и управление ресурсами

Процесс - экземпляр программы во время исполнения, единица активности ОС, в которой существуют последовательные действия, текущее состояние и набор связанных ресурсов

Процесс включает:

- исполняемую программу
- один или несколько потоков
- адресное пространство
- контекст исполнения (регистры, РС и т.д.)
- связанные структуры ядра
- ресурсы (память, файлы, устройства)
- контекст безопасности
- динамические библиотеки

Проблемы:

- **Защита памяти** — изоляция процессов, чтобы не портили память друг друга
- **Взаимоблокировки** — deadlock / livelock / starvation
- **Неправильная синхронизация** — гонки/потеря сигналов (race condition)
- **Сбой взаимного исключения** — одновременный доступ к общему ресурсу без mutex/semaphore

Планирование выполнения процессов и управление ресурсами

Равноправие

ОС стремится равномерно распределять ресурсы между процессами и пользователями. Достигается планировщиками.

Дифференциация отклика

Интерактивные задачи получают больший приоритет для минимизации задержек (например, окно в фокусе).

Общесистемная эффективность

Минимизация простоев, максимальная загрузка CPU и ресурсов.

Планировщики

- планировщик процессов
- планировщик дисковых операций
- разные классы диспетчеризации:
 - Time Sharing

- Real Time
- Fair Share

8. Управление памятью, виртуальная память. Защита информации и безопасность ОС.

Управление памятью

- **Изоляция процессов** — у каждого процесса своё виртуальное адресное пространство.
- **Выделение/освобождение** — ОС даёт и забирает память у программ.
- **Поддержка модулей** — можно подключать/отключать и менять размер областей памяти.
- **Защита доступа** — права r/w/x на память, запрет чужого доступа.
- **Долговременное хранение** — хранение данных на диске после выключения.
- **Страничный обмен** — перенос страниц между RAM и диском (swap).

Виртуальная память

Виртуальная память — механизм, при котором каждый процесс работает в своём **виртуальном адресном пространстве**, а ОС через **таблицы страниц** отображает виртуальные страницы в физические фреймы → **изоляция** и защита.

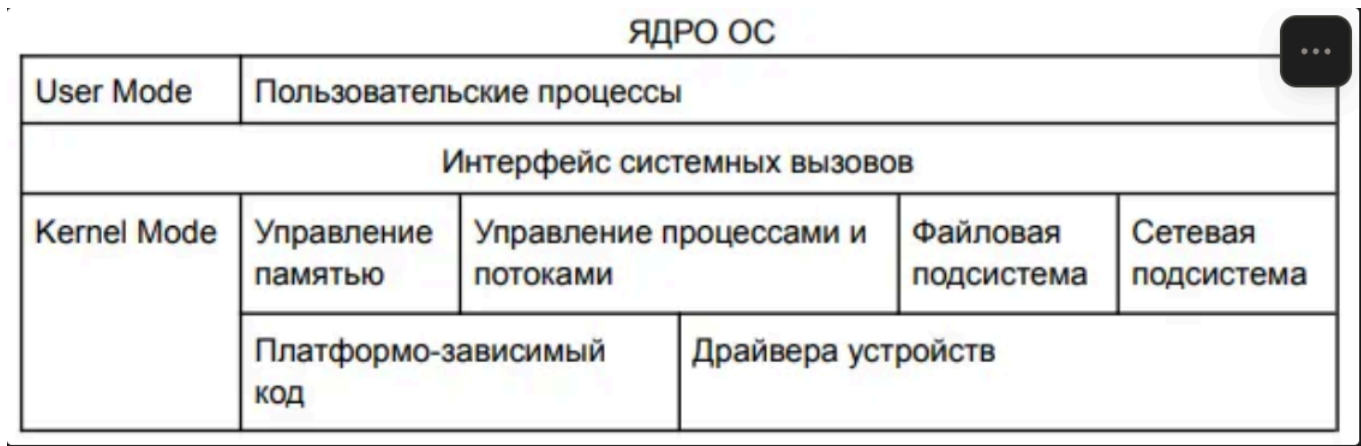
При обращении к адресу: **VA** → **(TLB)** → **таблицы страниц** → **РА + смещение**.

Если нужной страницы нет в RAM, возникает **page fault**, ОС подгружает её (из swap или из файла) и продолжает выполнение.

Защита информации и безопасность ОС

- **Доступ к системе** — защита от несанкционированного входа.
- **Конфиденциальность** — недопуск утечки данных.
- **Целостность данных** — защита от несанкционированного изменения/повреждения.
- **Аутентификация** — кто ты (проверка личности).
- **Авторизация** — что тебе можно (права/доступ).

9) Структура ядра операционной системы. Архитектуры монолитного ядра, ядра динамически загружаемыми модулями и микроядра.



1) Монолитное ядро

Это ядро, где основные подсистемы ОС работают в режиме ядра и в одном адресном пространстве.

2) Модульное ядро

Это тот же монолит, но с возможностью подключать части ядра как модули (обычно драйверы, ФС, сетевые штуки).

- Можно **загрузить/выгрузить модуль** без пересборки ядра (иногда даже без перезагрузки).
- Ядро остаётся монолитным по архитектуре: модули всё равно работают **в Kernel Mode**.

3) Микроядро

Микроядро оставляет в **Kernel Mode** только самое минимальное:

- планирование (или часть планирования)
- управление памятью (минимум)
- IPC (межпроцессное взаимодействие)
- базовая обработка прерываний

А остальное выносится в **user space сервисы**:

- драйверы
- файловые серверы
- сетевые сервисы и т.д.

10) Потоки исполнения, многопоточность, модели многопоточности

Поток - единица диспетчеризации и выполнения ОС (контекст процессора и часть стека)

Процесс - один или несколько потоков (связанные ресурсы)

Многопоточность - технология где процесс разделяется на несколько одновременно выполняемых потоков. Полезно для приложений выполняющих несколько независимых заданий не требующих последовательного выполнения

POSIX Threads - библиотека порождения потоков

Модели многопоточности:

One to One: на каждый user поток есть kernel поток, большие накладные расходы

One to Many: все потоки сопоставляются с одним потоком ядра и управляются на уровне пользователя без вмешательства ОС

Many to Many: несколько пользовательских потоков сопоставляются с меньшим кол-во ядра, баланс между гибкостью и производительностью

11) Симметричная и асимметричная многопроцессорная обработка

AMP (Asymmetric Multiprocessing) — асимметричная

Идея: есть **главный процессор (Master)** и **подчинённые (Slave)**.

Master принимает решения и управляет системой, а Slave выполняют выделенные задачи.

Примеры использования:

- CPU + GPU: CPU подготавливает данные, GPU обрабатывает
- Старые встроенные или специализированные системы

Недостатки:

- Мастер-процессор - потенциальная точка отказа
- Сложность масштабирования и низкая устойчивость
- Меньше гибкость в распределении нагрузки

SMP (Symmetric Multiprocessing) — симметричная

Идея: все процессоры/ядра **равноправны**, работают под **одной ОС** и имеют общий доступ к памяти.

ОС сама распределяет процессы/потоки между ядрами.

Особенности:

- Каждый процессор может выполнять любую задачу, в том числе системную
- ОС автоматически распределяет процессы и потоки между процессорами
- Поддержка вытесняющей многозадачности и многопоточности

Архитектурные реализации:

- UMA
- NUMA

12) Виртуализация. Типы виртуализации

Виртуализация

Виртуализация — технология, которая **абстрагирует вычислительные ресурсы** (CPU, память, диск, сеть) от их физической реализации и позволяет **изолированно запускать несколько сред** на одном физическом компьютере.

Основные типы

Виртуальные машины (VM) + гипервизор

Пример: **VirtualBox, KVM, Hyper-V, VMware**

- На одном ПК запускаются **несколько ОС**
- Каждая VM имеет **своё ядро**, драйверы и системные службы

Контейнеризация

Пример: **Docker, LXC**

- Изоляция **на уровне ОС** (процессы, FS, сеть)

Виртуальная машина языка (runtime VM)

Пример: **JVM, .NET, Python VM**

- Программа выполняется через **байткод/интерпретатор**

Облачные технологии

- системы построенные на базе аппаратной виртуализации, с дополнительными настройками прав доступа и общего мониторинга, выделение ресурсов

13) Сбои и отказоустойчивость ОС. Причины появления отказов в ОС и способы борьбы с ними

Отказ или сбой - наблюдаемое проявление дефекта, в том числе падение программы. Может привести к невозможности выполнить задачу или получить верный результат.

Отказоустойчивость - способность системы продолжать работу при аппаратных или программных ошибках.

Причины отказов системы

- **Аппаратные:** поломка CPU/диска/ОЗУ, перегрев, скачки напряжения.
- **Программные:** баги ОС/драйверов/приложений (утечки, гонки, краши), повреждение данных.
- **Человеческий фактор:** ошибки администратора, неверная настройка, случайное удаление/изменение.
- **Внешние:** атаки, проблемы питания, сбои сети/окружения

Способы борьбы с отказами

- **Резервирование (избыточность):** дублирование/тройное (2 устройства вместо 1, либо “голосование” 2 из 3).
- **Hot-swap:** “горячая” замена компонентов без остановки системы.
- **Поддержка ОС:** обнаружение сбоев, отключение/переподключение устройств, перенос нагрузки.
- **RAID:** отказоустойчивое хранение данных в дисковой подсистеме.

14) Надежность. Среднее время восстановления. Коэффициент доступности и время простоя

Надежность (Reliability) - способность системы сохранять работоспособность в течение заданного времени в определенных условиях эксплуатации

MTTF (Mean Time To Failure)

Среднее время до отказа — ожидаемое время работы устройства/системы до первого отказа (чаще для невозстанавливаемых).

MTTR (Mean Time To Repair)

Среднее время восстановления — сколько в среднем нужно, чтобы вернуть систему в рабочее состояние после отказа (Обнаружение сбоя → Диагностику → Устранение →

Перезагрузку ОС).

Меньше MTTR → выше доступность системы.

Доступность - вероятность, что система находится в рабочем состоянии

Время простоя (downtime) — интервал с момента неработоспособности сервиса до момента возобновления его работы.

15) Резервирование и отказоустойчивость

Резервирование - создание дублирующих компонентов систем в случае отказа

Отказоустойчивость - способность системы продолжать функционировать при отказе

Методы резервирования

- **Физическая (пространственная) избыточность:** дублирование компонентов/узлов, резерв включается при отказе (пример: запасной DNS/сервер).
- **Временная избыточность:** повтор операции при ошибке (ретрансляция пакета/блока данных).
- **Информационная избыточность:** избыточные данные/коды для обнаружения и исправления ошибок (ECC-память, коды/избыточность в RAID).

Методы повышения отказоустойчивости

- **Изоляция:** разделение процессов/ресурсов, чтобы сбой не “ронял” всё.
- **Управление параллелизмом:** корректные блокировки/борьба с дедлоками.
- **Виртуализация:** сильнее изоляция и легче перенос/резервирование.
- **Checkpoint/rollback:** точки восстановления и откат к сохранённому состоянию после сбоя.

16) История и развитие ОС GNU/Linux. Single UNIX Specification и POSIX

История

- 1983 - Ричард Столлман объявляет о создании проекта GNU - свободной юникс подобной системы
- 1991 - Линус Торвальдс публикует ядро Linux
- Так и родилась ОС GNU/Linux, состоящая из ядра линукс и пользовательского окружения GNU

POSIX - набор стандартов для обеспечения совместимости между различными UNIX системами. Системные вызовы, инструменты командной строки, потоки, сигналы

межпроцессорного взаимодействия.

SUS - стандарт от The Open Group для стандартизации UNIX систем, включает в себя POSIX, но дополняет его более строгими требованиями к ядру операционной системы, файловой системы и утилитам, так же сетевые функции международная поддержка и расширенные api

17) Понятие дистрибутива, дистрибутивы Linux.

Дистрибутив - комплекс файлов и программных компонентов, собранных и сконфигурированных так, что их по сути можно использовать “из коробки”.

Ну или... готовый к установке пакет ПО, включающий:

- Ядро Linux
- Системные утилиты GNU (bash, coreutils, glibc и тп)
- Систему управления пакетами (пакетный менеджер - apt, rpm...)
- Графическую подсистему
- Дополнительное ПО (текстовые редакторы, браузеры, офисные приложения)
- Конфигурационные файлы и инструменты установки

Дистрибутивы делятся на коммерческие и community

- **Community** не имеют поддержки и развиваются сообществом, например: debian/ubuntu, fedora/centos arch gentoo
- **Коммерческие** имеют поддержку и как правило платные: red hat, suse, oracle, astra

18) Архитектура и основные подсистемы Linux. Linux Kernel Map.

Архитектура

Архитектура ядра Linux представляет собой набор взаимосвязанных подсистем, каждая из которых отвечает за определённую часть работы операционной системы.

Основные подсистемы Linux

- **Планировщик задач** — управляет процессами/потоками, переключает контекст, распределяет время CPU “справедливо”.
- **Виртуальная память** — адресные пространства, страницы, mmap, page fault, paging/swapping.
- **Физическая память** — управление RAM (страницы/аллокаторы), связь с виртуальной через **MMU** (Memory Management Unit).

- **Файловая подсистема (VFS)** — единый интерфейс для разных ФС, монтирование в одну иерархию, структуры **inode/dentry**.
- **Сетевая подсистема** — стек TCP/IP (IPv4/IPv6, UDP), сокеты, обработчики протоколов, драйверы, фильтрация трафика.
- **Устройства и драйверы** — **блочные** (диски/SSD: буферизация) и **символьные** (терминалы: поток байтов), работа через `/dev`.
- **Прерывания и исключения** — обработка IRQ/exception: **top-half** быстро, **bottom-half** отложено (softirq/workqueue).
- **IPC** — сигналы, shared memory/семафоры/очереди сообщений, сокеты (обмен между процессами).
- **Синхронизация** — spinlock, mutex, semaphore, rwlock, seqlock, RCU; в user-space важен **futex**.

Linux Kernel Map

Все эти подсистемы вместе образуют карту ядра Linux (Linux Kernel Map) и обеспечивают выполнение программ, управление ресурсами и защиту системы

19) История и развитие Windows

Windows/MS-DOS появился как расширение примитивной операционной системы MS-DOS которая успешно пользовалась на ранних ПК.

После история развития шла так:

- **DOS-линия:** 1 → 2 → 3.x → 95 → 98 (→ ME)
(ещё не полностью “современная” архитектура, много наследия DOS)
- **NT-линия (полноценное современное ядро):** NT → 2000 → XP → Vista → 7 → 8 → 10 → 11
Версии 1 и 2 были надстройкой над DOS

В первой значимой версии 3.11 ещё не была развита многозадачность, так что она ещё не была полнофункциональной ОС

А вот 95 и 09 уже были

Также есть ответвления (сервера, мобилки, IoT)

20) Общая архитектура Windows. Windows API

Общая архитектура Windows

Вообще архитектурно винда похожа на юникс. Есть процессы, которые могут быть пользовательскими и служебными, есть сервисы, приложения, окружения. Есть

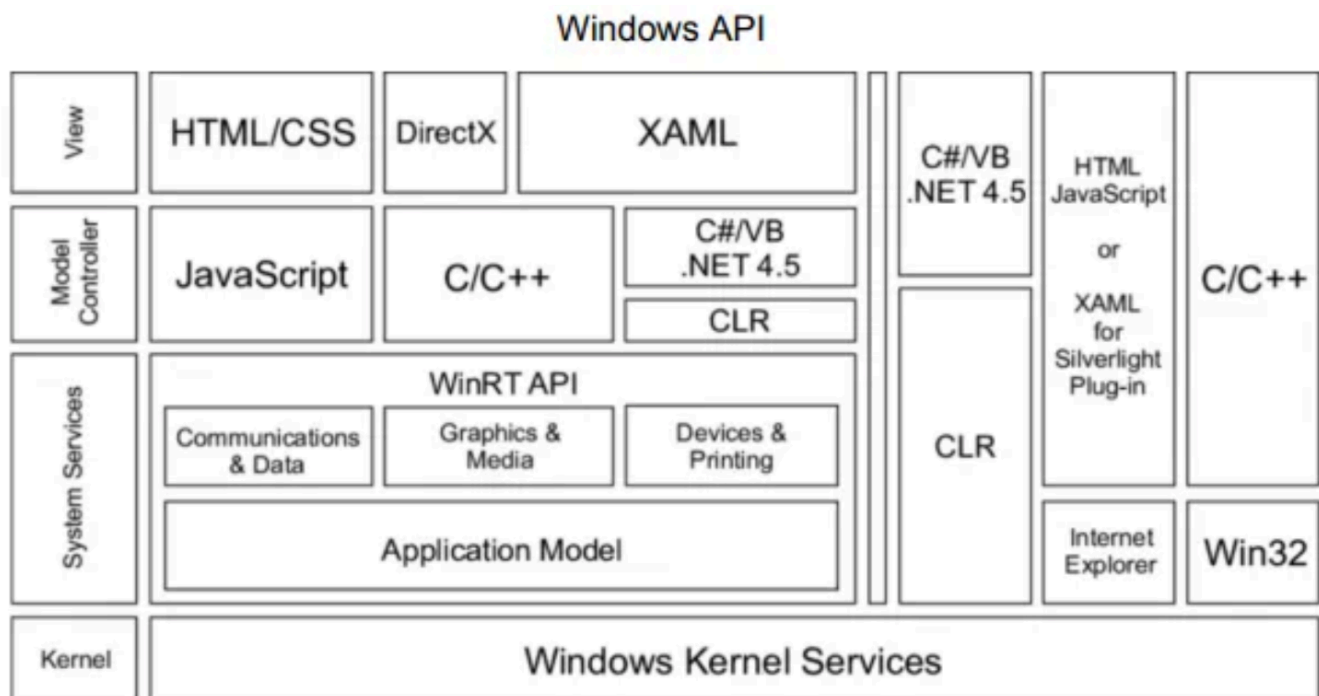
интерфейс системных вызовов - ntdll.dll. Эти вызовы работают в kernel mode. Виртуальная память тоже схожа. Имеются подсистемы управления (энергией, виртуальной памятью, кешем, процессами и потоками). Интерфейс для единообразного взаимодействия с аппаратурой - HAL (hardware abstraction layer).

Services/System Processes - службы, которые стартуют при входе в виндоус и останавливаются и удаляются, когда пользователь выходит. ОС содержит множество процессов, которые присутствуют каждый раз, когда мы загружаем наши машины и эти процессы отвечают за кучу всего от создания интерфейса и инициализации до загрузки драйверов и DLL.

Environment Subsystems - окружения, которые эмулируют те или иные работы ОС. Например эмулирующие позикс, чтобы запустить прогу, написанную под линукс (извлечение).

Важная особенность винды - в отличие от юникса, графическая подсистема, а именно графические драйверы и оконная система, являются частью ядра.

Windows API



Windows Kernel Services — ядро, которое управляет процессами, памятью, вводом-выводом и драйверами

System Services — системные службы ОС. На них построены два основных прикладных интерфейса:

- **Win32** (классический API для старых desktop-приложений)

- **WinRT** (современный API с **Application Model** и наборами функций для данных, графики/медиа, устройств)

Приложения используют эти API через разные технологии и языки:

- **C/C++** напрямую
- либо **C#/VB** через **CLR**
а интерфейс может быть **XAML/HTML/DirectX**.

21) Сервисы, функции и важные компоненты Windows.

Сервисы и функции

- **Windows API functions (WinAPI)** — высокоуровневые функции для приложений (создать процесс/файл: `CreateProcess`, `CreateFile`).
- **System calls (Native System Services)** — системные вызовы, которые напрямую переводят выполнение в ядро (низкоуровневый интерфейс NT).
- **Kernel support functions** — внутренние функции ядра/драйверов для служебных операций (например выделение памяти в ядре).
- **Windows services** — фоновые службы ОС, выполняют системные задачи и предоставляют функциональность приложениям.
- **DLL (Dynamic Link Libraries)** — динамические библиотеки с готовыми функциями, которые подключаются программами для расширения возможностей.

Важные компоненты системы

- **Hyper-V** — встроенный гипервизор Windows для запуска виртуальных машин.
- **Firmware** — подсистема загрузки микрокода/прошивок для устройств при старте.
- **Terminal Services** — удалённые пользовательские сессии и работа через “тонкие клиенты”.
- **Объекты и безопасность** — ресурсы ОС представлены объектами, доступ контролируется правами и политиками.
- **Реестр** — централизованное хранилище конфигурации системы и программ.
- **Оснастки (MMC и др.)** — админ-инструменты для управления системой (службы, устройства, политики).

22) Процесс, характеристики процесса в момент выполнения. Состояние процесса. Разделение ресурсов

Процесс

Процесс - выполняемая программа / экземпляр программы выполняемый на компьютере / единица активности характеризующаяся выполнением последовательных команд текущим состоянием и связанным с ней множеством системных ресурсов

Характеристики процесса (в момент выполнения)

- **PID** (идентификатор)
- **состояние** (выполняется / готов / ждёт / завершён)
- **приоритет**
- **счётчик команд (PC) и регистры** (контекст)
- **адресное пространство** (код/данные/стек/общие области)
- **I/O состояние** (открытые файлы, устройства, запросы)
- **учёт** (CPU time и др.)

Состояния процесса

- **New** (создан / создание)
- **Ready** (готов)
- **Running** (выполнение)
- **Blocked / Waiting** (ожидание)
- **Terminated / Exit** (завершён)

Переходы происходят из-за кванта времени, I/O, ожидания ресурса или завершения.

23) Модель процесса с пятью состояниями, назначение состояний

Основные состояния процесса

- **Выполняется (Running)** — процесс сейчас на CPU.
- **Готов (Ready)** — может быть запущен при освобождении CPU.
- **Блокирован / Ожидает (Blocked/Waiting)** — ждёт событие (I/O, ресурс).
- **Новый (New)** — создан, но ещё не включён в пул готовых.
- **Завершён (Terminated)** — завершил работу, ресурсы освобождаются

24) Paging и Swapping. Модель процесса с семью состояниями.

Paging и Swapping

- **Swapping** — выгрузка **всего процесса целиком** на диск. Использовалось раньше, сейчас редко из-за большого размера процессов.

- **Paging** — выгрузка **отдельных неиспользуемых страниц памяти**. Страницы при необходимости подгружаются обратно; код можно перечитать из файла, данные — из swap. Ядро не выгружается.

Модель процесса с 7 состояниями

К базовым состояниям (**New, Ready, Running, Blocked, Terminated**) добавляются:

- **Ready/Runnable Suspend** — процесс готов к выполнению, но частично или полностью находится в swap.
- **Blocked/Wait Suspend** — процесс ждёт событие и выгружен на диск.

Эти состояния нужны, чтобы ОС могла **освобождать память**, не теряя контроль над процессами.

25) Управляющие таблицы процесса. Образ процесса.

Ядро ведёт управляющие структуры для основных подсистем: **память, файлы, устройства, процессы**.

Для каждого процесса есть **PCB (Process Control Block)** — запись, где хранится состояние процесса и ссылки на его ресурсы (память, открытые файлы, I/O и т.д.).

Образ процесса

Образ процесса - это layout его адресного пространства. Сюда входят и пользовательские элементы, и ядерная часть (да, это я уже писала, но повторение...). При запуске части образа берутся из исполняемого файла, а части создаются как отдельные участки памяти. При этом разные сегменты обладают разными правами и могут быть разделяемыми между процессами.

Элементы образа процесса:

- Данные пользователя (допускающая изменения часть пользовательского адресного пространства, куда могут входить данные программы, пользовательский стек и модифицируемый код)
- Пользовательская программа (программа, которую надо выполнить, неизменяема)
- Системный стек (хранение параметров, адресов вызова процедур и системных служб)
- Управляющий блок процесса (данные, необходимые для управления процессом)

26) Управляющий блок процесса (PCB), состав PCB.

PCB (Process Control Block) - структура данных, используемая ОС для хранения всей информации о процессе. Создается при инициализации процесса и хранит следующую информацию:

- Идентификация процесса
- Информация о регистрах (необходима для переключения процессов)
- Другая информация, управляющая процессом (см. страшную картинку)

Можно сказать, что самая важная структура данных в ОС, т.к. информация из этих блоков читается и изменяется почти каждым модулем ОС (включая модули планирования, распределения ресурсов, обработки прерываний, контроля и анализа).

PCB должен храниться в памяти, защищенной от обычного доступа процесса. В некоторых ОС этот блок размещается в начале стека ядра процесса

27) Функции ОС, связанные с процессами. Создание процесса, переключение процессов.

Функции ОС, связанные с процессами

- **Управление процессами**
 - Создание и завершение процессов
 - Планирование и диспетчеризация процессов
 - Переключение процессов
 - Синхронизация и поддержка обмена инфой между процессами
 - Организация управляющих блоков процессов
- **Управление памятью**
 - Выделение адресного пространства процессам
 - пейджинг и свопинг
 - управление страницами и сегментами
- **Управление вводом-выводом**
 - Управление буферами
 - Выделение процессам каналов и устройств ввода-вывода
- **Функции поддержки**
 - Обработка прерываний
 - Учет использования ресурсов
 - Текущий контроль системы

Создание процесса

- Присвоить процессу уникальный идентификатор
- Выделить процессу память
- Инициализировать PCB
- Поставить процесс в очереди ядра

- Создать потоки ввода-вывода
 - Создать другие управляющие структуры данных
- В Unix-like системах для этого комбинация `fork` (копирование процесса) и `exec` (замена образа на новый). В винде `CreateProcess`.

Переключение процессов

Переключение (context switch) происходит при переходе в **kernel mode**: ОС сохраняет состояние текущего процесса (регистры, РС, стек) и затем восстанавливает состояние другого.

Переход в ядро обычно вызывают:

- **таймерное прерывание** (квант времени закончился → планировщик выбирает другой процесс),
- **I/O прерывание** (событие завершилось → ожидавшие процессы переводятся в ready),
- **page fault** (нужной страницы нет в RAM → процесс блокируется, страница подгружается, потом процесс снова ready),
- **системный вызов** (trap: приложение просит сервис ОС, например открыть файл).

28) Процессы в ОС UNIX SVR4. Диаграмма состояний, основные структуры

Процессы в UNIX SVR4: состояния и логика

В **SVR4** процесс может выполняться в **user mode** и **kernel mode**, что отражено состояниями **user running** и **kernel running**.

Переход в ядро происходит через **syscall**, **прерывания**, **page fault**, после чего возможна **диспетчеризация** и переключение на другой процесс.

Состояния процесса

- **Created** — процесс создан (`fork`), но ещё не готов к выполнению
- **Ready (mem)** — готов и находится в оперативной памяти
- **Ready (swpd)** — готов, но выгружен в swap
- **User running** — выполняется пользовательский код
- **Kernel running** — выполняется код ядра
- **Preempted** — вытеснен планировщиком
- **Sleep (mem)** — ждёт событие в памяти
- **Sleep (swpd)** — ждёт событие и выгружен в swap
- **Zombie** — завершён, но запись о процессе ещё не собрана родителем

Наличие состояний **mem / swpd** отражает активное использование **swapping**.

Основные структуры

PCB в Solaris состоит из `proc_t` и `user_t` и занимает около 2 Кб памяти. `p_as` - указатель на адресацию памяти. Содержит структуры, которые относятся к адресному пространству и его конкретным сегментам, а так же определяют, именованное оно (сегмент данных в файле) или анонимное (куча, стек). Так же содержит треды на уровне ядра и пользователя

29) Понятие потока выполнения, связь потока и процесса. Преимущества потоков.

Понятие потока выполнения

Поток - единица исполнения кода, на уровне ОС связанная с планированием и диспетчеризацией, в отличие от процесса, который является единицей владения ресурсами.

У каждого потока есть свой контекст (регистры, состояние выполнения), стек ядра, стек пользователя и набор локальных переменных, которые “умирают” вместе с потоком, но адресное пространство и остальные ресурсы процесса для всех потоков в процессе общие.

Связь процесса и потока

Процесс — это **контейнер ресурсов** (адресное пространство, файлы, права), общий для всех потоков.

Потоки выполняются **внутри процесса**: у каждого свой контекст и стеки, но память и ресурсы общие, поэтому нужна **синхронизация** при доступе к данным.

Преимущества потоков

- **Дешёвое создание и завершение** по сравнению с процессами
- **Быстрое переключение контекста** (общее адресное пространство)
- **Быстрый обмен данными** через общую память
- **Параллелизм** на многоядерных системах
- **Гибкое управление приоритетами** (фон/реактивные задачи)
- **Модульность и асинхронность** (I/O и сервисные задачи в отдельных потоках)

30) Состояния потока, User Level Threads vs Kernel Level Threads

Состояния потока

У потока те же базовые состояния, что и у процесса: Новый, Готов к выполнению, Выполняется, Ожидание (блокирован) и Завершен.

Потоки переходят между состояниями под управлением планировщика: из готов в выполняется при выделении кванта, в ожидание при блокирующем вводе-выводе, обратно в готов после завершения и в завершен при окончании работы, после чего структуры освобождаются.

User Level Threads vs Kernel Level Threads

- **ULT (Green threads)** - потоки уровня юзера реализуются библиотекой в пользовательском пространстве.
 - Переключение между ними выполняется без перехода в ядро, что делает переключения очень дешевыми. ОС при этом может вообще не знать о существовании этих потоков и видеть только один процесс, а либа имитирует планировщик в юзер спейсе и сама планирует их.
- **KLT (Kernel)** - потоки уровня ядра известны ядру - на каждый пользовательский поток создается соответствующий поток ядра, который планируется обычным планировщиком ОС
- Гибрид - несколько потоков выступают как исполнители, на которые либа отображает большее кол-во пользовательских потоков. Это позволяет уменьшить кол-во потоков ядра и часть переключений делать в юзер спейсе, но оставляет возможность реального параллелизма

31. Многопроцессорность и многопоточность. Закон Амдала.

Многопроцессорность и многопоточность

Поток - единица параллельности: разные потоки одного процесса могут реально выполняться на разных процессорах/ядрах, если ОС планирует их на разные CPU. При одном процессоре потоки дают удобную структуру программы, при нескольких процессорах - еще и реальное ускорение, потому что параллельные части работы выполняются одновременно, а не по очереди.

И в целом многопоточность нужна именно для того, чтобы использовать многопроцессорность.

Закон Амдала

На сколько можно ускорить программу на N процессорах с использованием потоков? Ответ на этот вопрос дается в законе Амдала, показывающем, что ускорение от добавления процессоров ограничено долей параллельного кода

$$\text{Ускорение} = \frac{\text{Время работы на одном процессоре}}{\text{Время выполнения на } N \text{ процессорах}} = \frac{1}{(1-f) + \frac{f}{N}}$$

- f — параллельная часть, $1-f$ — последовательная.
- При маленьком f параллелизм неэффективен

32. Механизм параллельных вычислений, функции ОС.

Механизм параллельных вычислений

Механизм параллельных вычислений - набор средств ОС, которые дают процессу возможность запускать несколько потоков/процессов на исполнение одновременно и координировать их работу.

Параллельность достигается за счет наличия нескольких потоков управления, которые ОС может планировать на разные процессоры или по очереди на одном, создавая иллюзию одновременности

Функции ОС в поддержке параллельности

- **Учёт ресурсов** — ОС отслеживает, какие ресурсы кем заняты.
- **Распределение/освобождение** — выдаёт ресурсы по запросу и корректно освобождает их.
- **Защита ресурсов** — изоляция от воздействия других процессов/потоков.
- **Детерминированность** — результат не зависит от скорости выполнения и конкуренции.

33. Проблемы параллельного выполнения: взаимного исключения, взаимоблокировки, голодание. Требования к взаимным исключениям. Уровни взаимодействия процессов и потоков.

Проблемы параллельного выполнения:

- Взаимного исключения - процессы/потоки не должны одновременно использовать критический ресурс
- Взаимоблокировки - процессы/потоки не должны взаимозахватывать требуемые ресурсы

- Голодание - конкуренция за ресурсы не должна порождать невозможность доступа к ресурсу

Требования к взаимным исключениям

- Взаимоисключения осуществляются в принудительном порядке
- Процесс/поток не должен влиять на другие процессы/потoki в не критическом участке
- Противодействие бесконечному ожиданию доступа к критическому участку
- Вход в свободный критический участок должен незамедлительно предоставляться
- Отсутствие предположений о кол-ве процессов или их скорости
- Ограничение времени нахождения в критических участках

Уровни взаимодействия процессов и потоков

- **Не осведомлены друг о друге** — независимые процессы, не предназначены для совместной работы; конкурируют за ресурсы.
Проблемы: взаимоблокировки, взаимное исключение, голодание.
- **Косвенно осведомлены** — совместно используют общий объект (буфер, файл, I/O), без знания ID друг друга.
Проблемы: взаимное исключение, взаимоблокировки, голодание, порча общих данных.
- **Непосредственно осведомлены** — созданы для совместной работы и общаются по идентификаторам (IPC).
Проблемы: взаимоблокировки, голодание (взаимоисключение контролируется).

34. Примитивы синхронизации ОС. Предназначение примитивов синхронизации

Примитивы синхронизации ОС

- **Семафор** — счётчик для синхронизации и управления доступом к ресурсу (P/V); бывает счётный и бинарный.
- **Мьютекс** — взаимное исключение; блокировать и разблокировать может **только** владелец.
- **Условная переменная** — ожидание выполнения условия (sleep/wakeup), используется вместе с мьютексом.
- **RW-lock** — несколько читателей или один писатель.
- **Монитор** — высокоуровневая конструкция: критические секции + очередь ожидания, один активный поток.
- **Флаги событий** — ожидание одного или набора событий по битам.

- **Сообщения (IPC)** — обмен сообщениями между процессами, может служить синхронизацией.
- **Spinlock** — активное ожидание блокировки (busy-wait), эффективно при коротких критических секциях.

Предназначение примитивов синхронизации

- обеспечить **взаимное исключение** при доступе к общим ресурсам
- **синхронизировать выполнение** процессов и потоков
- предотвратить **гонки данных (race condition)**
- координировать **ожидание и пробуждение** по событиям
- обеспечить **корректность и детерминированность** параллельного выполнения

35. Примитивы синхронизации ОС. Семафоры и мьютексы. Бинарный семафор

Семафоры

Семафоры - захват и освобождение множественного ресурса или одного в случае бинарных семафоров. Блокирующий примитив.

“proberen” (проверка)

- уменьшаем счетчик ресурса и проверяем. Если счетчик меньше нуля - блокируем поток в очереди, иначе допускаем к ресурсу

“verhogen”

- когда ресурс освобождается потоком, проверяем, есть ли в очереди потоки. Если да - запускаем поток из очереди.

Потоки в очереди именно блокируются, поэтому это дорого и медленно (сначала переключить контекст при блокировании, затем вернуть обратно)

Мьютексы

Мьютексы — примитив взаимного исключения: **один поток владеет ресурсом и только он может его освободить** (lock/unlock). Используются для защиты критических секций.

Реализации:

- **Блокирующие** — поток засыпает при занятости.
- **Spin** — активное ожидание в цикле.
- **Адаптивные** — сначала spin, если владелец выполняется, иначе sleep.
- **Futex** — мьютекс в user space с редким входом в ядро, снижает накладные расходы.

36. Примитивы синхронизации ОС. Условные переменные, rwlocks.

Условные переменные

Условные переменные — примитив синхронизации для **ожидания события/условия**: поток блокируется, пока другой поток не подаст сигнал.

- **wait()** — отпускает мьютекс и засыпает; при пробуждении снова захватывает мьютекс.
- **signal/notify()** — будит **один** ожидающий поток.
- **broadcast/notifyAll()** — будит **всех** ожидающих.

Важно: условная переменная **всегда используется вместе с мьютексом**, который защищает данные условия.

Блокировки чтения/записи (RW-lock) rwlocks

Блокировки чтения/записи (RW-lock) — позволяют **нескольким потокам читать одновременно**, но **писать только одному**.

- **readlock()** — общий доступ: допускается много читателей.
- **writelock()** — эксклюзивный доступ: ждёт, пока уйдут все читатели, затем пишет один поток.
- После записи ресурс снова открыт для чтения.

Проблема - когда ресурс освобождается, все потоки хотят максимально быстро захватить ресурс

37. Примитивы синхронизации ОС. Мониторы, флаги событий, передача сообщений.

Мониторы

Монитор — высокоуровневый примитив синхронизации (обычно на уровне ЯП), который скрывает мьютексы/условные переменные. Пример: `synchronized` в Java.

- доступ к общим данным только через **процедуры монитора**
- **в один момент времени** внутри монитора работает **только один поток** (взаимное исключение)
- ожидание/сигналы делаются через **condition variables** (`wait/notify`)

Флаги событий

Флаги событий (event flags) — синхронизация через набор битов: поток ждёт, пока нужный **флаг/комбинация флагов** станет 1.

Операции:

- **set** (установить)
- **clear** (сбросить)
- **wait** (ждать флаг)
- **wait any** (ждать любой из набора)
- **wait all** (ждать все)

Передача сообщений

Почтовые ящики (message passing) — высокоуровневый примитив обмена сообщениями, часто реализуется как **очередь** и может использоваться для синхронизации.

- операции: **send()** / **receive()**
- адресация:
 - **прямая** (по отправителю/получателю)
 - **косвенная** — через идентификатор ящика (1:1, 1:N, M:1, M:N)
- синхронизация: **блокирующая** / **неблокирующая**, возможна гарантия доставки
- сообщения: фиксированной или переменной длины
- выборка из очереди: **FIFO**, **приоритет**
- приём: от конкретного отправителя или любого

38. Примитивы синхронизации ОС. Неплокирующие примитивы синхронизации и неплокирующие структуры данных.

Неплокирующие примитивы синхронизации

Блокировка вызывает переключение контекста. Это долго и дорого. Идея неплокирующих алгоритмов: стараться не вызывать блокировку вовсе, а организовать синхронизацию так, чтобы поток не засыпал и не требовал переключения контекста.

Типы прогресса:

- **Wait-free** — каждый поток всегда завершается за конечное число шагов (самая сильная гарантия).
- **Lock-free** — система в целом продвигается: хотя бы один поток делает прогресс.
- **Obstruction-free** — поток завершится, если ему не мешают другие.

К неблокирующим примитивам относятся различные аппаратные атомарные операции и спинлок, как пример по сути неблокирующего примитива, потому что поток крутится на месте, не засыпая

Неблокирующие структуры данных

К неблокирующим структурам данных относятся: очереди, деревья и тп, для которых придуманы алгоритмы вставки и удаления без классических блокировок. Их реально достаточно много тк это была модная тема.

39. Управление памятью, основные определения и требования к организации.

Основные определения:

- **Основная память (RAM)** - область, где процессор может исполнять программы
- **Вторичная память** - область, где программы и данные могут храниться, в том числе и во время выполнения
- **Кадр (frame)**- область фиксированного размера основной памяти
- **Страница** - область фиксированного размера во вторичной памяти, которая может быть скопирована в кадр
- **Сегмент** - область переменного размера в основной или вторичной памяти, может быть разделена на страницы

Требования к организации памяти

- **Переместимость** — программа должна работать независимо от того, где она размещена/перемещена.
- **Защита** — изоляция процессов, контроль прав доступа (аппаратно).
- **Совместное использование** — управляемый shared-доступ (общий код/библиотеки/SHM) без потери защиты.
- **Логическая организация** — поддержка модулей (код/данные/стек), удобное адресное пространство.
- **Физическая организация** — управление обменом RAM ↔ диск (paging/swapping), потому что RAM ограничена и дорога, диск медленнее, но большой и постоянный.

40. Фиксированное и динамическое размещение программ в памяти.

Фиксированное

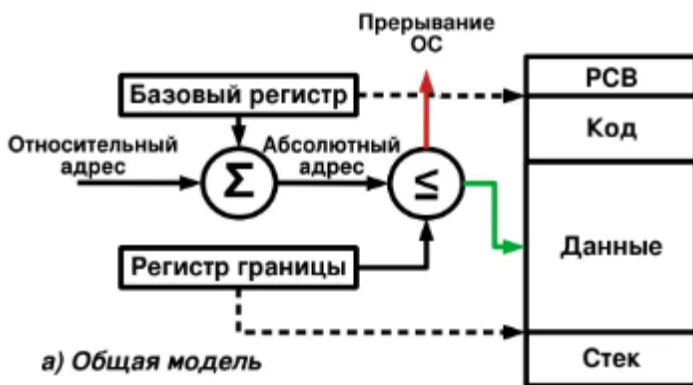
Фиксированное размещение программ в памяти - основная память разделяется на ряд статических разделов во время генерации системы. процесс может быть загружен в раздел равного или большего размера

Динамическое

Динамическое размещение - разделы создаются динамически. Каждый процесс загружается в раздел строго необходимого размера

41. Модели аппаратного перемещения программ

- Модели аппаратного перемещения:



Классический случай:

- базовый регистр - указывает на начало программы
- регистр границы - конец программы (иногда вместо него указывается длина сегмента)
- Программа оперирует **смещением** (относительным адресом).
- Железо делает: **физический = база + смещение**.
- Если адрес **вылез за лимит** → **прерывание** (защита памяти).

Сегментная модель 8086

логический адрес = сегментный регистр + смещение, где сегментный регистр задает начало сегмента. Сегментный регистр (16 бит) сдвигается влево на 4 бита, затем к нему прибавляется 16ти битное смещение, так получается 20ти битный физический адрес и адресуемые ~64Кб памяти

- Есть разные сегментные регистры (**CS/DS/SS/ES**) для кода/данных/стека и т.д., что упрощает размещение частей программы.

42. Простой страничный подход и простая сегментная организация.

Simple paging (простой страничный подход):

- Память делят на **кадры** фиксированного размера, процесс — на **страницы** того же размера.
- **Логический адрес = (номер страницы, смещение)** → по таблице страниц находим кадр → **физический адрес = кадр + смещение**.
- **Внешней фрагментации нет**, остаётся только **внутренняя** (обычно в последней странице процесса).
- В памяти одновременно держим процессы по **количеству свободных кадров**, размещение гибче, чем при разделах.

Simple segmentation (простая сегментная организация):

- Адресное пространство процесса делится на **сегменты переменного размера** (код, данные, стек и т.п.).
- **Логический адрес = (номер сегмента, смещение)** → по таблице сегментов берётся база и длина, проверяется выход за границы.
- **Внутренней фрагментации нет**, но возможна **внешняя фрагментация** (хотя меньше, чем при разделах).

43. Виртуальная память основные определения и принципы организации аппаратуры и управляющих программ.

Определения:

- **Физический адрес** - адрес ячейки в основной памяти. Память видится как линейный блок адресов.
- **Виртуальный адрес** - логический адрес внутри процесса. Процессы видят одинаковые адресные пространства, но реально они попадают в разные физические адреса
- **Виртуальное адресное пространство** - область для одного процесса с виртуальными адресами
- **Виртуальная память** - схема расположения процессов в памяти, в которой:
 - Вторичная память адресуется так же как основная
 - Виртуальные адреса транслируются в адреса основной памяти
 - Основная память может быть расширена на вторичную
 - Размер памяти ограничен схемой адресации, но не фактическим количеством ячеек
- **Резидентная часть процесса (RSS)** - та часть процесса, которая сейчас реально находится в физической памяти. Остальное лежит в файлах и области подкачки.

Организация

Организация виртуальной памяти - совокупность аппаратных и программных средств

Схемы реализации:

- совокупность страничной и сегментной организации
- логические адреса динамически транслируются в физические
- сегменты и страницы не должны располагаться последовательно в основной памяти
- любая часть процесса в разные моменты выполнения может находиться во вторичной памяти и менять физические адреса в основной

Разные структуры организации:

- одноуровневая
- двух- и многоуровневая
- инвертированная

Таблицы страниц хранятся в основной памяти. Для обращения к памяти нужно сделать несколько служебных обращений. Используется TLB для ускорения.

44. Виртуальный страничный обмен. Двухуровневая организация MMU и TLB 80386. (для КОТ и ГТ — общие принципы)

Формат виртуального адреса

Виртуальный адрес (32 бита) делится на три части:

- **PDE (10 бит)** — индекс в каталоге страниц (Page Directory),
- **PTE (10 бит)** — индекс в таблице страниц (Page Table),
- **Offset (12 бит)** — смещение внутри страницы (страница 4 КБ).

Основные структуры

- **CR3** — регистр, указывающий на начало каталога страниц (PDT) текущего процесса.
- **Каталог страниц (PDT)** — содержит указатели на таблицы страниц.
- **Таблицы страниц (PT)** — содержат номера физических кадров.
- **TLB** — кэш в процессоре, хранящий последние преобразования виртуальных страниц в физические кадры.

Алгоритм трансляции адреса

1. MMU берёт виртуальный адрес и сначала ищет соответствие в **TLB**.

- Если запись есть → сразу получаем номер физического кадра + смещение → физический адрес.

2. Если записи в TLB нет (TLB miss):

- Из регистра **CR3** берётся адрес каталога страниц.
- По полю **PDE** выбирается нужная таблица страниц.
- По полю **PTE** выбирается физический кадр.
- К номеру кадра прибавляется **Offset**.
- Полученный результат — физический адрес.
- Соответствие добавляется в TLB.

Виртуальный страничный обмен (Page Fault)

Если при обращении в каталоге страниц или таблице страниц бит *Present* = 0, это означает, что страница отсутствует в оперативной памяти.

Тогда возникает **Page Fault**:

- major fault - страница реально читается с диска во фрейм
- minor fault - страница уже в памяти, но нет нужного маппинга, ядро только достраивает структуры без обращения к диску

В 80386 используется двухуровневая страничная адресация: виртуальный адрес состоит из индекса каталога страниц, индекса таблицы страниц и смещения.

Регистр CR3 указывает на каталог страниц процесса.

По PDE выбирается таблица страниц, по PTE — физический кадр, к которому прибавляется смещение.

Для ускорения используется TLB — кэш трансляций адресов.

При отсутствии страницы в памяти возникает page fault, и ОС подгружает её с диска.

45. Инвертированная таблица страниц.

Инвертированная таблица страниц — это способ организации таблицы страниц, при котором хранится не отображение

- «виртуальная страница → физический кадр»,
а наоборот —
- «**физический кадр → (PID, номер виртуальной страницы)**».

То есть таблица одна на всю систему, и каждая её запись соответствует одному кадру физической памяти и содержит:

- идентификатор процесса (PID),
- номер виртуальной страницы этого процесса,
- биты состояния (present, dirty и т.п.).

Поиск идет так: вычисляется хэш -> берется нужный бакет -> затем по цепочке сравниваются виртуальные адреса, пока не найдется нужный маппинг или не станет ясно, что его нет

46. Сегментно-страничная виртуальная память

Сегментно-страничная организация памяти объединяет преимущества сегментации и страничной адресации.

Виртуальный адрес имеет структуру:

- **номер сегмента – номер страницы – смещение.**

Алгоритм:

- По номеру сегмента в таблице сегментов определяется базовый адрес таблицы страниц данного сегмента и проверяется, что обращение не выходит за границы сегмента (защита).
- По номеру страницы в таблице страниц сегмента определяется номер физического кадра.
- К адресу кадра прибавляется смещение, в результате получается физический адрес.

47. Влияние размера страницы виртуальной памяти на ОС. Стратегии ОС по работе с виртуальной памятью.

Влияние размера страницы виртуальной памяти на ОС

Размер страницы в виртуальной памяти влияет на баланс между фрагментацией, числом страниц, нагрузкой на TLB и частотой page fault. ОС подбирает размер и стратегии так, чтобы минимизировать расходы и не убить систему свопами

- При **маленькой странице** меньше внутренняя фрагментация.
 - Но чем страница меньше, тем их больше в процессе, растет размер таблиц страниц и число обращений к ним, а соответственно чаще TLB miss, page fault, тк обрабатывать приходится больше мелких блоков.
- При **крупной странице** уменьшится число записей в TLB и таблицах, что снижает накладные расходы и число промахов.
 - Но растет внутренняя фрагментация. Процессу выделяют крупный фрейм, из которого реально используется только часть.

Стратегии ОС по работе с виртуальной памятью

1. Стратегия выборки страниц:

- По требованию (Demand Paging) — страница загружается при первом обращении.
- Предварительная (Prefetching) — ОС заранее подгружает соседние страницы.

2. Стратегия размещения:

- В NUMA-системах страницы размещаются ближе к тому процессору, который с ними работает.

3. Стратегия очистки (выгрузки):

- По требованию (при нехватке памяти).
- Фоновая (заранее освобождаются редко используемые страницы).

4. Стратегии замещения страниц:

- FIFO — первая загруженная вытесняется первой.
- LRU — вытесняется давно не использовавшаяся.
- LFU — вытесняется реже всего используемая.
- Clock — приближённая LRU.

48. Стратегии замещения страниц ОС. Часовой Алгоритм. Управление резидентной частью процесса.

Стратегии замещения страниц ОС. Часовой Алгоритм

Стратегии замещения страниц определяют, какую страницу выгрузить из оперативной памяти при возникновении page fault и отсутствии свободных кадров.

- Идеальной считается оптимальная стратегия, которая вытесняет страницу, к которой обращение произойдёт через наибольший промежуток времени, но она нереализуема, так как будущее поведение программы неизвестно.
- FIFO вытесняет самую старую загруженную страницу, проста в реализации, но может удалять активно используемые данные.
- LRU основана на принципе локальности и вытесняет страницу, не использовавшуюся дольше всего, однако требует сложных механизмов учёта обращений.
- На практике часто применяется часовой алгоритм (Clock) — приближённая реализация LRU, в которой каждая страница имеет бит обращения, а указатель по кругу выбирает страницу, давно не использовавшуюся.

Управление резидентной частью процесса

Резидентная часть процесса — это множество его страниц, находящихся в данный момент в оперативной памяти. ОС должна управлять её размером, так как при слишком малом размере резко возрастает число page fault, а при слишком большом уменьшается количество процессов, которые могут одновременно эффективно работать в памяти.

Поэтому применяются стратегии управления резидентным набором:

- с фиксированным размером
 - с динамическим
- когда ОС увеличивает или уменьшает число кадров в зависимости от поведения процесса. Замещение может быть локальным, когда процесс вытесняет только свои страницы, или глобальным, когда для освобождения памяти могут вытесняться страницы других процессов.

49. Виды планирования процессов. Критерии краткосрочного планирования. Приоритеты.

Виды планирования процессов

- **Долгосрочное планирование** - решение о добавлении процесса в пул выполняемых процессов
- **Среднесрочное планирование** - решение о добавлении процесса к числу процессов, полностью или частично размещенных в основной памяти
- **Краткосрочное планирование** - решение о том, какой из процессов будет выполняться процессором
- **Планирование ввода-вывода** - решение о том, какой из запросов процессов на операции ввода-вывода будет обработан доступным устройством ввода-вывода

Критерии краткосрочного планирования

Пользовательские критерии:

- время оборота (turnaround time) — от запуска до завершения процесса,
- время отклика (response time) — от запроса до первого ответа,
- соблюдение предельных сроков (deadline),
- предсказуемость выполнения.

Системные критерии:

- пропускная способность (throughput),
- загрузка процессора,
- справедливость (отсутствие голодания),
- учет приоритетов,
- сбалансированная загрузка ресурсов

Приоритеты

Приоритет отражает важность процесса и используется планировщиком для выбора, кого запускать первым и кому давать больше/меньше процессорного времени

- Фиксированные приоритеты - статические, почти не меняются
- Динамические - система поднимает приоритет интерактивным/ожидающим процессам и снижает тем, кто долго и жадно грузит проц

50. Использование приоритетов.

В операционной системе каждому процессу назначается числовой приоритет. Планировщик всегда выбирает для выполнения процесс с наивысшим приоритетом

Процессы с одинаковыми приоритетами помещаются в одну очередь, и внутри неё используется дополнительная стратегия планирования

При приоритетном планировании возможна проблема **голодания**: процессы с низким приоритетом могут долго не получать процессор, если постоянно появляются более приоритетные задачи.

Для борьбы с голоданием применяются:

- **динамические приоритеты (aging)** — приоритет процесса постепенно повышается по мере ожидания;
- **вытесняющее планирование** — если в системе появляется процесс с более высоким приоритетом, текущий процесс может быть прерван и заменён.

51. Стратегии планирования FCFS, RR, SPN, SRT, HRRN, Feedback.

FCFS (First Come First Served):

- Выбирается процесс, который дольше всех ждет в очереди

RR (Round Robin):

- Круговое планирование. У всех процессов квант времени, они по очереди получают процессор по этому кванту.

SPN (Shortest Process Next):

- Короткие процессы вперед. Выбирается процесс с наименьшим ожидаемым общим временем выполнения

SRT (Shortest Remainig Time):

- Вытесняющая версия SPN. Если появляется более короткий процесс, он вытесняет с исполнения текущий

HRRN (Highest Response Ratio Next):

- Функция выбора $R = (W+S)/S$, где W - время ожидания, S - ожидаемое время выполнения. Чем дольше процесс ждёт, тем выше его R .

Feedback (многоуровневая обратная связь):

Приоритет процесса динамически изменяется:

- если процесс долго использует CPU — его приоритет понижается,
- если часто блокируется на I/O или долго ждёт — приоритет повышается.

52. Feedback планировщик и классы планирования ОС UNIX SVR4.

Feedback планировщик

Feedback-планировщик (многоуровневый с обратной связью) — это планировщик с динамическими приоритетами.

Его идея такая:

- Все готовые процессы разбиты на несколько очередей с разными приоритетами.
- Процесс получает квант времени CPU.
- Если процесс **полностью выбирает квант** (долго крутится на CPU) → считается CPU-bound → его приоритет **понижается**.
- Если процесс **часто блокируется на I/O** (ждёт диск, клавиатуру, сеть) → считается I/O-bound → его приоритет **повышается**.
- Таким образом:
 - Короткие и интерактивные процессы быстро обслуживаются.
 - Длинные вычислительные процессы постепенно спускаются в очереди с меньшим приоритетом.
- Это приближение к LRU по CPU-времени и защита от голодания.

В SVR4 приоритет пересчитывается по формулам с параметрами:

- `globpri` — текущий глобальный приоритет
- `quantum` — квант времени
- `tqexp` — приоритет после полного использования кванта
- `slpret` — приоритет после ожидания

- **maxwait** — максимум ожидания перед принудительным повышением.

Классы планирования

- **TimeSharing (TS)** — основной пользовательский класс с feedback-алгоритмом; динамические приоритеты (~0–59).
- **Interactive (IA)** — TS с приоритетным бустом для активных GUI/терминальных процессов (лучший отклик).
- **Fixed / System / RealTime** — классы с **фиксированными приоритетами**; квант задан, приоритет не меняется.
- **Fair Share (FSS)** — распределяет CPU пропорционально заданным долям процессов/групп.
- **Interrupt threads** — отдельные приоритеты для потоков обработки прерываний.
- **NUMA affinity** — учёт привязки к узлам памяти для снижения задержек.

53. Справедливое планирование.

Справедливость - всем процессам из класса, претендующим на ресурс, предоставляется примерно одинаковый доступ.

Каждой группе процессов (например, каждому пользователю) назначается некоторое число долей — *shares*.

Доля процессорного времени, которую получает группа, равна отношению её *shares* к сумме *shares* всех групп. После этого процессы внутри одной группы уже делят между собой выделенную ей часть процессора.

Таким образом, процессорное время распределяется пропорционально заданным квотам между пользователями, а не пропорционально количеству их процессов, что обеспечивает справедливость и предотвращает захват ресурсов одним активным пользователем.

54. Планирование в многопроцессорных системах. Типы многопроцессорных систем с точки зрения организации планирования. Гранулярность и проектирование планировщиков процессов и потоков для многопроцессорных систем.

Типы многопроцессорных систем.

- Распределенные/кластерные системы
 - Набор систем со своей основной памятью и подсистемой ввода-вывода.
 - Распределение заданий между системами

- Распределение общих данных и результатов
- Функционально специализированные системы
 - Ведущий процессор выполняет координацию, его планировщик раздает подзадачи специализированным процессорам, которые сами по себе могут быть необщего назначения
 - Ведомые процессоры выполняют вычисления
- Сильносвязанные процессоры (SMP/NUMA)
 - Имеют общую память и систему кэшей
 - Управляются одной ОС

Гранулярность

Гранулярность - частота синхронизации между процессами в системе

- **Тонкая** — синхронизация очень часто (десятки инструкций).
- **Средняя** — синхронизация периодически (сотни инструкций).
- **Грубая** — редкая синхронизация между процессами.
- **Очень грубая** — синхронизация на уровне распределённых систем.
- **Независимая** — синхронизация не требуется, процессы не взаимодействуют

Проектирование планировщиков процессов и потоков для многопроцессорных систем

Назначение процессов CPU

- **Статическое (affinity)**: процесс “прибит” к одному ядру → лучше кэш-локальность, но возможен дисбаланс.
- **Динамическое (глобальная очередь)**: берём задачу из общей очереди на свободный CPU → просто, но есть конкуренция за очередь и миграции.
- **Динамическая балансировка**: у каждого CPU своя очередь + иногда “воровство/перенос” задач → компромисс между локальностью и балансом.

Использование многозадачности

- При большом числе CPU можно давать процессам **почти “свои” ядра**, уменьшая вытеснения и переключения, но риск **простоя ядер**, если нагрузку оценили плохо.

Подходы к планированию

- **Load Sharing**: глобальная очередь потоков → равномерно, но **локи очереди + промахи кэша**, хуже при тонкой/средней гранулярности.

- **Gang scheduling**: связанные потоки запускаются **одновременно на разных CPU** → меньше накладных расходов, полезно для синхронизируемых задач.
- **Dedicated processor assignment**: выделяем пул CPU под набор потоков → минимум переключений, но возможны **простои**.
- **Динамическое планирование**: если CPU не хватает, у процесса, который использует много CPU, **забирают часть** в пользу других.

55. ОС реального времени и планировщики. Deadline-планирование.

Операционные системы реального времени (RTOS) — это системы, в которых важна не только корректность результата, но и строгое соблюдение временных ограничений. Нарушение временных требований рассматривается как ошибка

Hard real-time - все задачи должны завершаться строго в заданные сроки; пропуск дедлайна недопустим (авиация, автоматика, медицина).

Soft real-time - небольшие задержки допустимы, но ухудшают качество работы (мультимедиа, сети).

Планировщики - имеют следующие свойства:

- вытесняемость: более важная задача немедленно вытесняет менее важную,
- строгая приоритетная схема,
- минимальные и детерминированные задержки,
- гарантированное время реакции,
- отсутствие голодания критических задач.

Deadline-планирование

Deadline-планирование — это метод планирования, при котором каждая задача описывается не только приоритетом, но и временными параметрами:

- время готовности (ready time)
- крайний срок начала (start deadline)
- крайний срок завершения (completion deadline)
- время выполнения (processing time)
- требуемые ресурсы

Планировщик выбирает к выполнению задачу так, чтобы все дедлайны были соблюдены.

Главная цель deadline-планирования — гарантировать своевременное выполнение всех критических задач даже при высокой нагрузке системы.

56. Проблема инверсии приоритетов, типы инверсии и способы решения в планировщике.

Инверсия приоритетов

Инверсия приоритетов — это ситуация, когда высокоприоритетный процесс не может выполняться, потому что он ждёт ресурс, захваченный низкоприоритетным процессом, а процесс со средним приоритетом в это время вытесняет низкоприоритетный и занимает процессор. В итоге получается, что высокий приоритет фактически обслуживается хуже, чем средний — порядок приоритетов «переворачивается».

Типы инверсий

- Ограниченная - есть процесс с высоким приоритетом и ему нужен ресурс, который занят процессом с более низким приоритетом. По хорошему мы должны дождаться завершения работы процесса и по окончании захватить ресурс
- Неограниченная - приоритетные потоки вытесняют менее приоритетные. Для решения этой проблемы придумано наследование приоритета, те поток, которому нужен ресурс, дает свой приоритет потоку на ресурсе и другим потокам сложнее его захватить
- Цепочка блокировок - передача уровня приоритета всей цепочке потоков с запросом на ресурс.

Способы решения

1. Наследование приоритета (Priority Inheritance)

Если высокоприоритетный процесс ждёт ресурс, захваченный низкоприоритетным, то низкоприоритетный временно получает приоритет высокого. Это позволяет ему быстро выполниться и освободить ресурс.

2. Потолок приоритета (Priority Ceiling Protocol)

Каждому ресурсу заранее назначается максимальный возможный приоритет. Процесс, захватывающий этот ресурс, автоматически поднимается до этого уровня, что предотвращает появление инверсии и цепочек блокировок.

57. Ввод-вывод. Современные устройства и скорости обмена, развитие способов ввода-вывода, логическая структура ввода-вывода.

Устройства ввода-вывода делятся на три группы:

1. **Пользовательские** — клавиатура, мышь, экран, принтер.
2. **Внутренние** — диски (HDD, SSD), видеокарта, контроллеры.
3. **Устройства связи** — сетевые карты, модемы, Wi-Fi, Ethernet.

Они сильно различаются по:

- скорости обмена (от байт/с до гигабайт/с),
- сложности управления,
- размеру передаваемых блоков,
- способам представления данных,
- обработке ошибок.

Развитие способов ввода-вывода

- **Программируемый ввод-вывод (PIO)**
 - Процессор напрямую управляет контроллером и ждёт завершения операции в цикле.
 - CPU простаивает → крайне неэффективно.
- **Ввод-вывод с прерываниями**
 - Контроллер посылает прерывание по завершении операции.
 - CPU может выполнять другие задачи, но копирование данных всё ещё делает сам
- **DMA (Direct Memory Access)**
 - Контроллер сам копирует данные между устройством и оперативной памятью.
- CPU только:
 - настраивает передачу,
 - запускает,
 - получает прерывание о завершении.

Логическая структура ввода-вывода

В ОС ввод-вывод разделён на уровни:

- **Логический ввод-вывод**
 - Процессы работают с абстракциями: файлы, сокеты, устройства (read, write, open).
- **Модуль устройства (драйвер)**
 - Преобразует логические операции в команды контроллеру.
- **Планирование и управление**

- Очереди запросов, прерывания, буферизация, синхронизация.
- **Аппаратный уровень**
 - Контроллеры, DMA, реальные устройства.

58. Буферизация ввода вывода. Ввод-вывод в UNIX SVR4.

Буферизация ввода-вывода нужна из-за сильного различия скоростей процессора и устройств ввода-вывода. Процессор работает намного быстрее дисков, сети, терминалов, поэтому если передавать данные напрямую, процессор будет простаивать, ожидая устройство. Для сглаживания этой разницы между устройством и процессом вводится промежуточная область в оперативной памяти — **буфер**

- **Смысл буферизации:**
данные сначала помещаются во временное хранилище (буфер), а уже затем передаются процессу. Пока процесс обрабатывает данные из буфера, устройство может параллельно загружать туда следующую порцию данных. Это позволяет перекрывать вычисления и ввод-вывод и повышает производительность.

Различают следующие схемы:

1. Небуферизованный ввод-вывод (Direct I/O)

Данные передаются напрямую между устройством и памятью процесса (обычно через DMA). Это быстро, но опасно для виртуальной памяти: страницы процесса приходится фиксировать в ОЗУ, поэтому используется только в специальных случаях (например, в СУБД).

2. Одиночная буферизация

Данные сначала поступают в буфер ядра, затем копируются в адресное пространство процесса. Пока процесс обрабатывает данные, устройство может готовить следующую операцию.

3. Двойная буферизация

Используются два буфера. Пока процесс читает данные из одного буфера, устройство заполняет второй. Это позволяет почти полностью перекрыть ввод-вывод и вычисления.

4. Кольцевая буферизация

Применяется для потоковых устройств (терминал, сеть, звук). Буфер организован в виде кольца: устройство последовательно записывает данные, процесс последовательно их считывает. Классическая схема producer–consumer.

В **UNIX SVR4** для блочных устройств (диски) используется **буферный кэш (BIO)**: данные хранятся в буферах ядра и переиспользуются, что уменьшает число обращений к диску.

Для символьных устройств применяются небольшие символьные буферы, а для высокопроизводительных задач возможен прямой ввод-вывод через DMA.

59. Диски и дисковое планирование

Дисковые устройства бывают двух основных типов: HDD и SSD.

HDD — это магнитный диск с вращающимися пластинами, дорожками и секторами, и с механической головкой чтения/записи. Время доступа к данным состоит из ожидания в очереди, перемещения головки к нужной дорожке (seek), ожидания поворота диска и передачи данных. Поэтому для HDD очень важно минимизировать движения головки.

SSD — это твердотельный накопитель на флеш-памяти, без движущихся частей. Доступ к ячейкам осуществляется электронно, поэтому нет задержек на позиционирование и вращение, и случайный доступ почти так же быстр, как последовательный.

Дисковое планирование нужно для того, чтобы операционная система определяла порядок обслуживания запросов к диску от разных процессов, стремясь:

- уменьшить общее время ожидания,
- повысить пропускную способность,
- обеспечить справедливость.

Базовые алгоритмы (без геометрии)

- **FIFO**: справедливо, но плохо по времени (много лишних перемещений).
- **Приоритетный**: быстрее для важных задач, но возможна деградация/голодание низких.
- **LIFO**: локальность для “свежих” запросов, но старые могут голодать.

Геометрические алгоритмы (учитывают положение головки)

- **SSTF** — обслуживается запрос, ближайший к текущей позиции головки;
- **SCAN** (алгоритм лифта) — головка движется в одну сторону, обслуживая все запросы по пути, затем в обратную;
- **C-SCAN** — обслуживание только в одном направлении с быстрым возвратом;
- **N-step-SCAN** и **FSCAN** — модификации для уменьшения очередей и голодания.

60. Концепции RAID.

RAID (Redundant Array of Independent Disks) — это способ объединения нескольких физических дисков в один логический массив, который для операционной системы выглядит как единое устройство хранения.

Данные в RAID распределяются по дискам блоками (striping), могут дублироваться (mirroring) или дополняться паритетом. Один и тот же запрос может обслуживаться несколькими дисками одновременно. А нужно это все чтобы получить скорость, отказоустойчивость или и то, и другое

Существуют различные уровни RAID (RAID 0, 1, 2, 3, 4, 5, 6 и их комбинации), которые по-разному сочетают скорость, объём и отказоустойчивость.

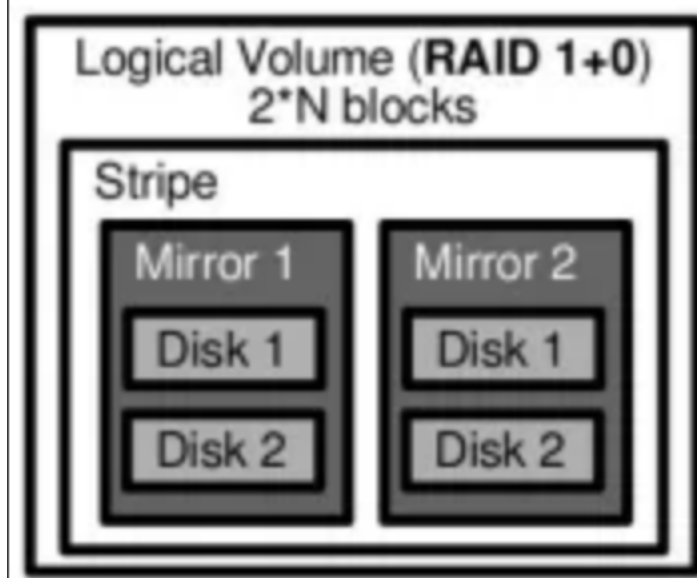
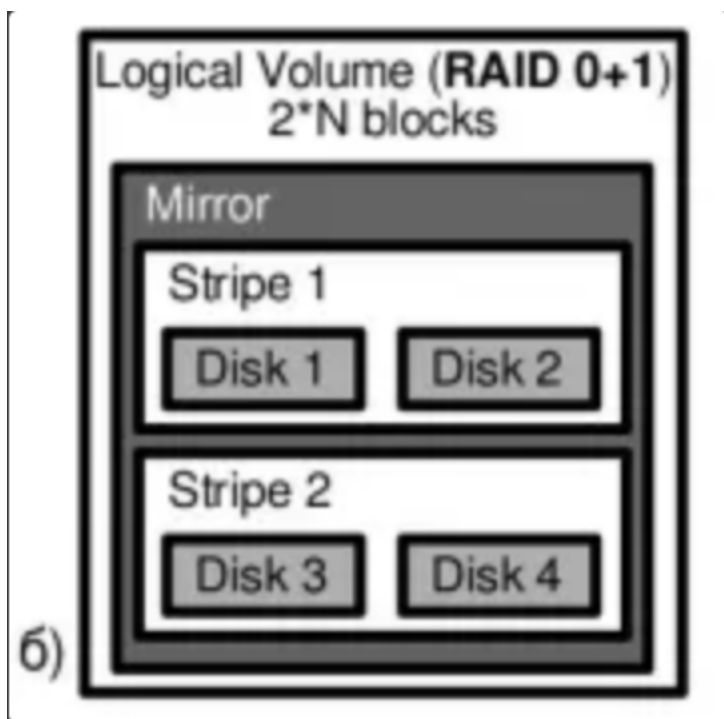
61. RAID-0, 1, 10, 0+1.

RAID 0 — это массив без избыточности. Данные разбиваются на блоки одинакового размера и распределяются по всем дискам (striping). За счёт параллельной работы дисков сильно возрастает скорость чтения и записи, а общий объём равен сумме объёмов всех дисков. Однако отказ одного диска приводит к потере всех данных.

RAID 1 — это зеркалирование. Все данные полностью дублируются на двух (или более) дисках. При отказе одного диска система продолжает работать с его копией. Чтение может быть быстрее за счёт параллельного доступа, но полезный объём равен объёму одного диска.

RAID 10 (1+0) — это комбинация RAID 1 и RAID 0, где сначала создаются зеркальные пары дисков (RAID 1), а затем по этим парам выполняется чередование блоков (RAID 0). В результате достигается высокая скорость и высокая надёжность: можно пережить отказ одного диска в каждой зеркальной паре.

RAID 0+1 — это зеркалирование массивов RAID 0. Сначала данные распределяются по дискам с помощью striping (RAID 0), затем весь полученный массив дублируется зеркалом. По скорости он похож на RAID 10, но по надёжности хуже, так как при отказе одного диска может выйти из строя целая половина массива.



62. RAID 4,5,6. Аппаратные дисковые массивы.

- RAID 4, RAID 5 и RAID 6 используют страйпинг данных и избыточность на основе чётности (parity).

RAID 4

- Данные разбиваются на страйпы и распределяются по дискам, а вся информация о чётности хранится на одном отдельном диске.
- Система выдерживает отказ одного диска, полезный объём равен $N-1$, но диск чётности становится узким местом при записи.

RAID 5

- Аналогичен RAID 4, но блоки чётности распределяются по всем дискам массива.
- Нет одного узкого диска чётности, повышается производительность записи.
- Также выдерживает отказ одного диска, полезный объём — $N-1$.

RAID 6

- Как RAID 5, но используется двойная чётность, вычисляемая по двум независимым алгоритмам.
- Система выдерживает отказ уже двух дисков одновременно, полезный объём — $N-2$, но скорость записи ниже из-за вычисления двух паритетов.

В аппаратных RAID-массивах все вычисления чётности, кэширование и восстановление выполняются специализированными контроллерами с собственной памятью и резервным питанием, что обеспечивает высокую скорость и отказоустойчивость.

63. Файловый ввод-вывод, основные определения. Задачи ОС по управлению файлами. Совместное использование файлов

- **Поле** — минимальная единица данных;
- **запись** — набор полей;
- **файл** — именованный объект долговременного хранения (в UNIX обычно просто поток байтов); **
- **БД**** — набор взаимосвязанных файлов/структур с отношениями.

Файлы бывают:

- **структурированные**
- **неструктурированные.**

Базовые операции:

- create/delete
- open/close
- read/write
- seek
- атрибуты
- блокировки

Задачи ОС:

- организация хранения на носителях
- именованное (каталоги)
- управление доступом
- надёжность/целостность (журналирование/синхронизация записи)
- производительность (буферизация/кэш)
- совместный доступ.

Совместное использование файлов обеспечивается:

- системой прав доступа (владелец, группы, разрешения),
- механизмами блокировок (всего файла или отдельных записей),
- синхронизацией процессов,

64. Управление файлами в UNIX SVR4

В UNIX SVR4 подсистема управления файлами построена многоуровневой.

- **Logical I/O (логический ввод-вывод).**

На этом уровне ОС работает с понятиями, понятными программисту: имена файлов, каталоги, права доступа, дескрипторы, операции open, read, write, close, lseek. Здесь происходит поиск файла по имени, проверка прав и сопоставление имени с открытым файловым дескриптором процесса.

- **VFS (Virtual File System).**

Это абстрактный слой, который делает все файловые системы одинаковыми для ядра. Он вводит единый объект — vnode (виртуальный файловый узел). Через него ядро выполняет операции над файлами независимо от того, в какой конкретной файловой системе они находятся (UFS, NFS, tmpfs и т.д.).

- **Конкретная файловая система (например, UFS в SVR4).**

Здесь vnode связывается с реальными структурами на диске — inode.

inode — это аналог PCB для процесса: он хранит всю метаданную о файле (тип, права, владельца, размер, времени, адреса блоков данных), но не имя.

- **Драйверы устройств и аппаратный уровень.**

Файловая система через драйверы выполняет реальные операции чтения и записи блоков на диск (через контроллеры, DMA, прерывания).

Таким образом, путь обращения к файлу выглядит так:

m

Процесс → Logical I/O → VFS (vnode) → конкретная ФС (inode, блоки) → драйвер → диск.

Эта архитектура позволяет:

- поддерживать разные файловые системы через единый интерфейс,
- изолировать работу с именами и правами от физической организации данных,
- обеспечивать переносимость и расширяемость ядра.

65. Каталоги файлов. Элементы каталога, операции ОС.

Каталоги файлов

Каталог — это специальный файл (структура данных), который хранит соответствие **имя файла → его inode (служебную запись)**.

Через inode ОС получает всю информацию о файле: расположение блоков на диске, размер, тип, владельца, права доступа, временные метки и т.д. Таким образом, каталог обеспечивает поиск файла по имени и организацию файлов в иерархию.

Элементы каталога

В каталоге (или через inode, на который он указывает) содержится:

- имя файла;
- тип файла;
- адресная информация (номер начального блока, размер);
- информация о владельце и правах доступа;
- информация об использовании (даты создания, чтения, изменения).

Структура каталогов

Каталоги образуют иерархию (дерево, а в UNIX — связный граф из-за ссылок). Корневой каталог является началом всей файловой системы.

Основные операции ОС над каталогами

Операционная система должна поддерживать:

- поиск файла по имени;
- создание записей (создание файлов и подкаталогов);
- удаление записей;
- получение списка файлов;
- изменение записей (переименование, изменение прав, владельца и т.п.);
- кэширование каталогов в памяти для ускорения доступа.

66. Размещение записей и файлов в блоках данных. Сложность и типы организации размещения.

Операционная система выполняет ввод-вывод не по записям, а по блокам фиксированного размера.

Запись — это логическая единица файла

блок — минимальная физическая единица передачи между диском и памятью.

Поэтому записи необходимо группировать внутри блоков.

Существует три основных способа размещения записей в блоках:

- **Фиксированное размещение (фиксированная длина записей).**
 - Все записи имеют одинаковый размер и целиком помещаются в один блок.
- **Переменная длина со сцеплением.**
 - Записи имеют разную длину и могут разрываться между блоками. Если запись не помещается целиком, её часть записывается в следующий блок, а между частями хранится ссылка.
- **Переменная длина без сцепления.**
 - Записи имеют разную длину, но каждая должна целиком помещаться в одном блоке. Если не помещается — используется следующий блок.

Размещение файлов (т.е. блоков файла на диске):

- **Непрерывное (contiguous):** блоки файла подряд
- **Связное (linked allocation):** каждый блок указывает на следующий
- **Индексное (indexed / inode):** есть индекс (таблица указателей на блоки)

Проблемы размещения: фрагментация (внутренняя, внешняя), частота выделения блоков, скорость доступа, размер служебной информации о размещении файлов

67. Непрерывное размещение файлов (на примере ОС RT-11)

Непрерывное (последовательное) размещение файлов — это способ, при котором все блоки файла располагаются подряд на диске.

Для описания файла достаточно хранить **номер первого блока и длину файла в блоках**.

Достоинства:

- **Очень высокая скорость чтения и записи**, так как блоки идут подряд и головка диска почти не перемещается.
- **Минимальный объём служебной информации** (только начало и длина).
- **Малая внутренняя фрагментация** — блоки в основном полностью заняты данными.

Недостатки:

- **Большая внешняя фрагментация**: при удалении файлов между занятыми областями появляются «дыры», и может не найтись достаточно большой непрерывный участок.
- **Сложно увеличивать файл**, так как рядом может не быть свободного места.
- Требуется **сжатие файловой системы** (перемещение файлов), чтобы устранить разрывы.

Итого:

Непрерывное размещение даёт **максимальную скорость**, но плохо масштабируется и страдает от внешней фрагментации

68. Цепочечное размещение файлов (на примере DOS FAT)

Цепочечное размещение — это способ хранения файлов на диске, при котором файл разбивается на блоки (кластеры), и эти блоки могут располагаться **в разных местах диска, не обязательно подряд**.

Связь между блоками хранится в специальной таблице — **FAT (File Allocation Table)**.

В ней для каждого кластера записано, какой кластер является следующим.

Таким образом файл представляется в виде цепочки:

кластер1 → кластер2 → кластер3 → ... → конец файла.

В каталоге хранится только номер первого кластера, а вся остальная цепочка восстанавливается по FAT.

Плюсы:

- Файл может динамически расти — новые кластеры просто прицепляются в конец цепочки.
- Нет внешней фрагментации в смысле «дырок» — можно использовать любые свободные блоки.

Минусы:

- Кластеры одного файла разбросаны по диску, поэтому при чтении головка HDD постоянно перемещается → падает скорость.
- Случайный доступ медленный, так как нужно проходить цепочку последовательно.
- При повреждении FAT теряется информация о размещении файлов.

Итог:

Цепочечное размещение уменьшает проблемы с выделением непрерывного места, но ухудшает скорость доступа из-за фрагментации и необходимости переходить по указателям.

69. Индексированное размещение (на примере файловой системы UNIX UFS).

Индексированное размещение файлов — это способ, при котором для каждого файла в файловой системе существует специальная структура данных — inode (индексный дескриптор).

Inode не является файлом, а является записью в служебной таблице файловой системы и содержит всю метаинформацию о файле (права доступа, размер, владельца, время создания и т.д.), а главное — **указатели на блоки данных файла**.

В UNIX в inode используются:

- прямые указатели на блоки данных (для небольших файлов),
- одинарно косвенный указатель (ссылка на блок указателей),
- двойной косвенный указатель,
- тройной косвенный указатель.

Такая многоуровневая система индексации позволяет размещать блоки файла в произвольных местах диска и эффективно работать как с малыми, так и с очень большими файлами, не требуя их непрерывного расположения.